

BUILD WEEK 3 // ESERCIZIO 4

Usare Wireshark per Esaminare il Traffico HTTP e HTTPS

Cattura, analisi e confronto tra
traffico in chiaro e traffico cifrato

Ubuntu VM (rete Bridged) | Wireshark + tcpdump

COLLETTIVO	NetRebels
CORSO	Cybersecurity & Ethical Hacking, A.A. 2025/2026
DATA	23 giugno 2026
AMBIENTE	Ubuntu (VM, rete Bridged) con Wireshark e tcpdump
STRUMENTI	tcpdump, Wireshark, Firefox, Terminale Bash
STATO	Completato



00

Indice

01 Introduzione

02 Obiettivi

03 Ambiente e Strumenti

04 Procedura

4.1 Parte 1: Catturare e visualizzare il traffico HTTP

4.2 Parte 2: Catturare e visualizzare il traffico HTTPS

05 Screenshot

06 Codice

07 Risultati

08 Difficoltà Incontrate

09 Conclusioni

01

Introduzione

In questo esercizio, noi di NetRebels abbiamo messo le mani sul traffico di rete per capire, una volta per tutte, perché un protocollo non vale l'altro. Dal nostro angolo di seminterrato pieno di cavi e monitor accesi a tutte le ore, abbiamo acceso Wireshark e tcpdump per intercettare e analizzare due tipi di comunicazione web: HTTP, il vecchio protocollo che manda tutto in chiaro come una cartolina senza busta, e HTTPS, la versione blindata che cifra ogni byte prima che lasci il dispositivo.

L'obiettivo non era solo tecnico, ma anche didattico: vedere con i nostri occhi cosa succede quando una password viene digitata in un sito HTTP, e confrontarlo con cosa (non) si vede quando lo stesso tipo di dato passa su un canale HTTPS. Una lezione pratica su quanto sia fragile la fiducia digitale quando manca la cifratura.

02

Obiettivi

Come riportato nella scheda dell'esercizio, i nostri obiettivi erano due, divisi in altrettante parti operative.

- **Parte 1:** Catturare e visualizzare il traffico HTTP
- **Parte 2:** Catturare e visualizzare il traffico HTTPS

Esercizio 4: Usare Wireshark per Esaminare il Traffico HTTP e HTTPS

Obiettivi

- **Parte 1: Catturare e visualizzare il traffico HTTP**
- **Parte 2: Catturare e visualizzare il traffico HTTPS**

Obiettivi dell'esercizio

03

Ambiente e Strumenti

Per condurre l'esercizio abbiamo lavorato all'interno di una macchina virtuale Ubuntu, con accesso a un terminale e al browser Firefox. Gli strumenti principali impiegati sono stati:

- **tcpdump**, per catturare il traffico di rete da riga di comando e salvarlo in un file .pcap
- **Wireshark**, per aprire il file di cattura e analizzare i pacchetti a livello di protocollo
- **Firefox**, per generare traffico reale visitando un sito HTTP e un sito HTTPS
- **Terminale Bash**, per identificare l'interfaccia di rete corretta con il comando `ip address`

La macchina disponeva di più interfacce di rete (`enp0s3` e `enp0s8`); abbiamo dovuto identificare quella effettivamente connessa a Internet prima di lanciare la cattura, per evitare di registrare un file vuoto o pieno di traffico irrilevante.

04

Procedura

- **Parte 1: Catturare e visualizzare il traffico HTTP**

STEP 1: Identificazione dell'interfaccia di rete

Abbiamo aperto un terminale e lanciato il comando `ip address` per elencare tutte le interfacce disponibili sulla VM. Dall'output abbiamo individuato `enp0s3`, con indirizzo IPv4 `10.0.2.15/24`, come interfaccia attiva verso la rete NAT della VirtualBox.

STEP 2: Avvio della cattura con tcpdump

Con l'interfaccia corretta in mano, abbiamo lanciato la cattura da riga di comando:

```
sudo tcpdump -i enp0s3 -s 0 -w httpdump.pcap
```

Il flag `-s 0` serve a catturare ogni pacchetto per intero (snappien illimitato, niente troncamenti), mentre `-w httpdump.pcap` salva tutto il traffico grezzo su disco, pronto per essere aperto in Wireshark.

STEP 3: Generazione del traffico HTTP

Con `tcpdump` in ascolto sullo sfondo, abbiamo aperto Firefox e visitato la pagina `vbsca.ca/login/login.asp`, una semplice pagina di test di login non cifrata. Abbiamo inserito username `admin` e una password di test, poi cliccato su Login.

STEP 4: Analisi in Wireshark

Fermata la cattura, abbiamo aperto `httpdump.pcap` in Wireshark e applicato il filtro `http` per isolare solo le richieste e risposte di interesse. Nella lista sono apparse, in ordine: la GET iniziale verso `/login/login.asp`, la risposta 200 OK, la POST verso `/login/login_results.asp` con i dati del form, e le risposte successive.

Selezionando il pacchetto POST, abbiamo espanso la sezione **HTML Form URL Encoded: application/x-www-form-urlencoded**, dove Wireshark ha decodificato in chiaro i campi inviati dal form: `txtUsername = admin` e `txtPassword = admin`.

Domanda: quali info vengono visualizzate nella sezione "HTML Form URL Encoded: application/x-www-form-urlencoded"?

In questa sezione Wireshark mostra ogni campo del form HTML inviato tramite il metodo POST, con il relativo nome (Key) e il valore digitato dall'utente (Value). Nel nostro caso compaiono `txtUsername` con valore `admin` e `txtPassword` con valore `admin`, entrambi leggibili in chiaro perché il protocollo HTTP non applica nessuna cifratura ai dati trasmessi.

- **Parte 2: Catturare e visualizzare il traffico HTTPS**

STEP 1: Nuova cattura per il traffico cifrato

Per la seconda parte abbiamo ripetuto la stessa logica della cattura, lanciando `tcpdump` su un nuovo file (`httpsdump.pcap`) sulla stessa interfaccia identificata in precedenza.

STEP 2: Generazione del traffico HTTPS

Abbiamo aperto Firefox e navigato verso `https://auth.netacad.com/auth/realms/skillsforall/login`, la pagina di accesso ufficiale di Cisco Networking Academy. A differenza della Parte 1, qui il lucchetto nella barra degli indirizzi confermava una connessione cifrata fin dal primo caricamento della pagina.

STEP 3: Analisi in Wireshark con filtro sulla porta 443

Aperto il nuovo file di cattura, abbiamo applicato il filtro `tcp.port==443` per isolare il traffico HTTPS. Selezionando un pacchetto di tipo TLS Application Data, abbiamo espanso la sezione **Transport Layer Security**, trovando un layer **TLSv1.2 Record** con Content Type "Application Data (23)" e un blocco di dati etichettato **Encrypted Application Data**, completamente illeggibile.

Domanda: cosa ha sostituito la sezione HTTP che era nel file di cattura precedente?

Al posto della sezione "Hypertext Transfer Protocol" troviamo ora la sezione **Transport Layer Security (TLS)**, che racchiude il record TLS con tipo di contenuto "Application Data" e il payload cifrato. Il protocollo applicativo reale (http-over-tls) è visibile solo come etichetta, non come contenuto leggibile.

Domanda: i dati dell'applicazione sono in formato plaintext o leggibile?

No. I dati sono mostrati come **Encrypted Application Data**, una sequenza di byte cifrati che non rivela in nessun modo url, parametri, username o password. A differenza della Parte 1, qui non esiste nessuna sezione equivalente a "HTML Form URL Encoded" da poter espandere.

Domanda: quali sono i vantaggi dell'uso di HTTPS invece di HTTP?

HTTPS aggiunge un livello di cifratura (TLS) sopra HTTP, garantendo tre cose che HTTP non offre: confidenzialità dei dati (nessuno in ascolto sulla rete, noi compresi, riesce a leggere username, password o contenuto delle richieste), integrità dei dati (un eventuale tentativo di alterare i pacchetti in transito viene rilevato), e autenticazione del server tramite certificato digitale (il browser verifica che il sito sia davvero chi dice di essere). Questo rende HTTPS molto più resistente ad attacchi di sniffing e man-in-the-middle rispetto al traffico in chiaro che abbiamo intercettato nella Parte 1.

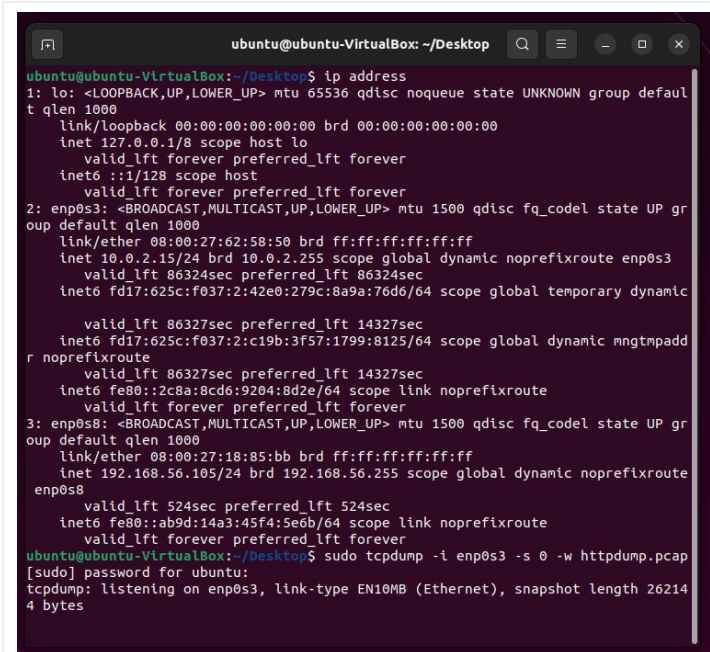
Domanda: tutti i siti web che usano HTTPS sono considerati affidabili?

No. HTTPS garantisce solo che il canale di trasmissione sia cifrato e che il traffico non possa essere letto o alterato facilmente durante il percorso, ma non dice nulla sull'affidabilità del contenuto o delle intenzioni di chi gestisce il sito. Un sito di phishing può benissimo avere un certificato HTTPS valido (oggi sono gratuiti e semplici da ottenere) e continuare a essere malevolo. Il lucchetto nella barra degli indirizzi protegge il "come" viaggiano i dati, non il "chi" li riceve.

05

Screenshot

Di seguito riportiamo gli screenshot raccolti durante l'esercizio, con la nostra lettura di ciascuno.

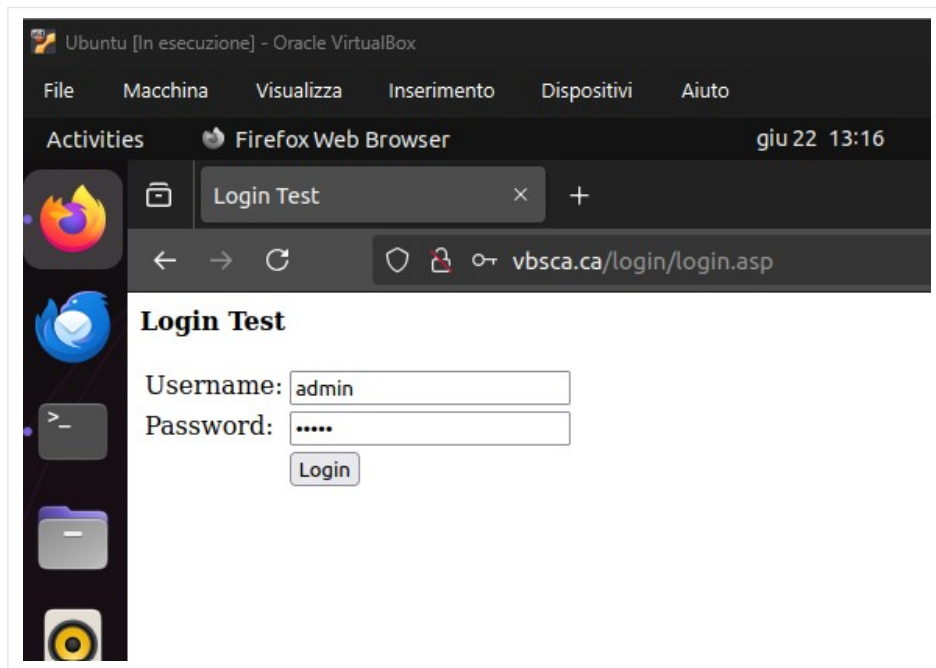
Identificazione interfaccia e avvio cattura

```
ubuntu@ubuntu-VirtualBox: ~/Desktop
ubuntu@ubuntu-VirtualBox:~/Desktop$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:62:58:50 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 86324sec preferred_lft 86324sec
    inet6 fd17:625c:f037:2:42e0:279c:8a9a:76d6/64 scope global temporary dynamic
        valid_lft 86327sec preferred_lft 14327sec
    inet6 fd17:625c:f037:2:c19b:3f57:1799:8125/64 scope global dynamic mngtnpaddr noprefixroute
        valid_lft 86327sec preferred_lft 14327sec
    inet6 fe80::2c8a:8cd6:9204:8d2e/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:18:85:bb brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.105/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s8
        valid_lft 524sec preferred_lft 524sec
    inet6 fe80::ab9d:14a3:45f4:5e6b/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
ubuntu@ubuntu-VirtualBox:~/Desktop$ sudo tcpdump -i enp0s3 -s 0 -w httpdump.pcap
[sudo] password for ubuntu:
tcpdump: listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

Terminale: ip address e avvio tcpdump

Nel terminale si vede l'output di `ip address`: l'interfaccia `enp0s3` risulta UP con IPv4 `10.0.2.15/24`, mentre `enp0s8` ha IP `192.168.56.105/24` (rete host-only, non rilevante per il traffico Internet). Subito sotto, il comando `sudo tcpdump -i enp0s3 -s 0 -w httpdump.pcap` viene lanciato e Wireshark/tcpdump confermano l'ascolto sull'interfaccia corretta, link-type EN10MB (Ethernet), snapshot length 262144 byte (cattura senza troncamento grazie a `-s 0`).

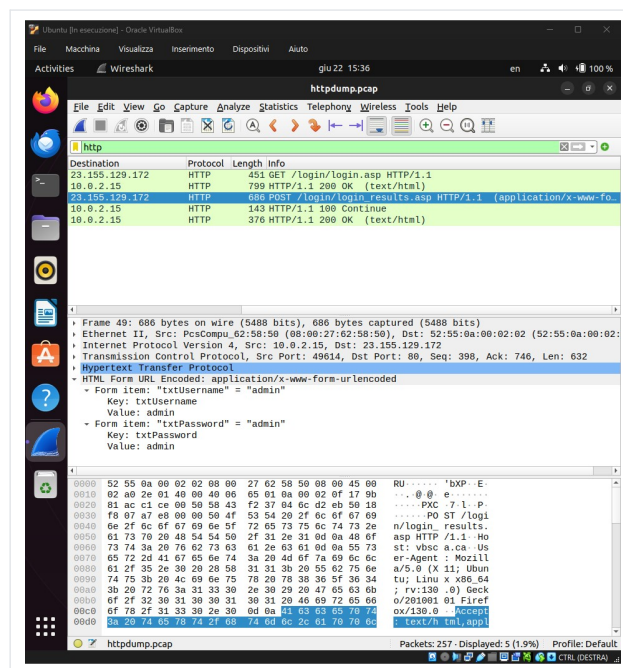
Form di login HTTP



Pagina di login HTTP su vbsca.ca

La pagina "Login Test" su vbsca.ca/login/login.asp mostra l'assenza del lucchetto nella barra degli indirizzi, segno che la connessione non è cifrata. Abbiamo inserito username admin e una password di sei caratteri prima di confermare il login.

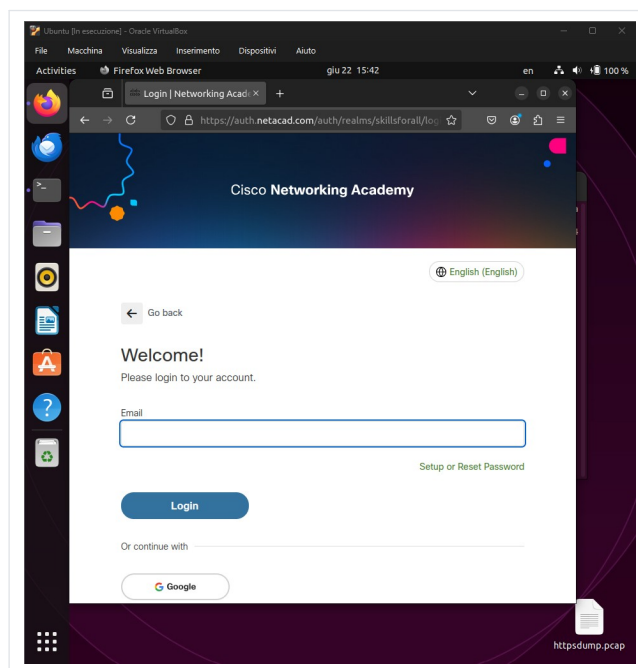
Pacchetto POST in Wireshark (HTTP)



Analisi del pacchetto POST in Wireshark

Con il filtro http applicato, la lista pacchetti mostra la sequenza GET, 200 OK, POST /login/login_results.asp, 100 Continue e un secondo 200 OK. Il pacchetto POST selezionato espone la sezione "HTML Form URL Encoded: application/x-www-form-urlencoded" con i campi txtUsername e txtPassword, entrambi con valore admin, leggibili anche nel pannello hex/ascii in basso.

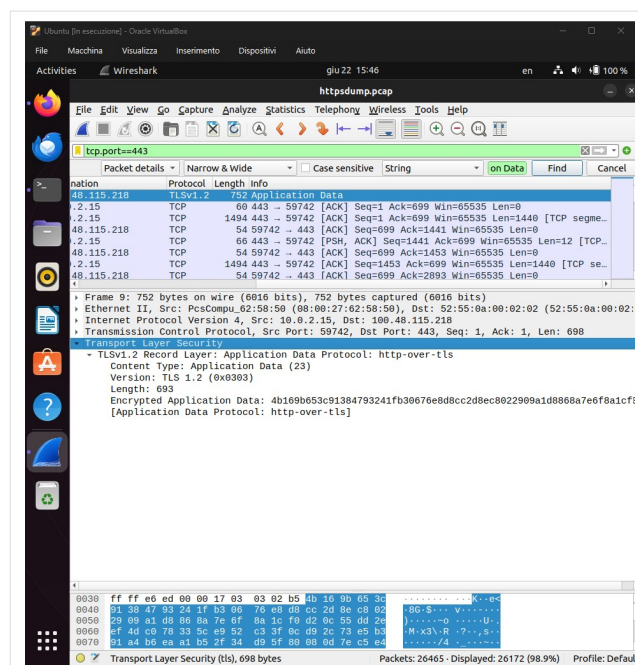
Form di login HTTPS



Pagina di login HTTPS su auth.netacad.com

Qui il lucchetto è ben visibile nella barra degli indirizzi di `https://auth.netacad.com/auth/realms/skillsforall/login`, a conferma che la sessione è protetta da TLS fin dal caricamento iniziale della pagina, prima ancora di inserire qualsiasi dato.

Pacchetto TLS Application Data in Wireshark (HTTPS)



Analisi del traffico TLS in Wireshark

Con il filtro `tcp.port==443`, la lista mostra pacchetti TLSv1.2 di tipo "Application Data" insieme ai normali ACK del TCP a supporto. Espandendo il pacchetto selezionato compare la sezione "Transport Layer Security" con il record TLSv1.2, Content Type "Application Data (23)" e il campo "Encrypted Application Data" seguito da una

lunga stringa esadecimale, senza alcun testo intelligibile.

06

Codice

Comandi utilizzati durante l'esercizio:

```
# Identificazione delle interfacce di rete disponibili
ip address

# Cattura del traffico HTTP sull'interfaccia enp0s3, salvato su file
sudo tcpdump -i enp0s3 -s 0 -w httpdump.pcap

# Cattura del traffico HTTPS sulla stessa interfaccia (Parte 2)
sudo tcpdump -i enp0s3 -s 0 -w httpsdump.pcap
```

Filtri applicati in Wireshark:

```
http
tcp.port==443
```

07

Risultati

Aspetto analizzato	Traffico HTTP	Traffico HTTPS
Protocollo visibile in Wireshark	Hypertext Transfer Protocol	Transport Layer Security (TLS)
Credenziali del form	Visibili in chiaro (admin / admin)	Non visibili, dati cifrati
Sezione dati applicativi	HTML Form URL Encoded	Encrypted Application Data
Indicatore visivo nel browser	Nessun lucchetto	Lucchetto presente
Rischio in caso di sniffing	Alto, credenziali esposte	Basso, payload illeggibile

La cattura ha confermato in modo netto la differenza tra i due protocolli: nella Parte 1 siamo riusciti a leggere username e password della form di test senza nessuno strumento di decifratura, semplicemente espandendo un campo di Wireshark. Nella Parte 2, lo stesso tipo di analisi si è fermata davanti a un blocco di byte cifrati, a dimostrazione pratica del perché oggi HTTPS sia lo standard de facto per qualsiasi sito che gestisca dati sensibili.

08

Difficolta Incontrate

La difficolta principale e stata identificare correttamente l'interfaccia di rete giusta prima di avviare tcpdump: la VM esponeva due interfacce (enp0s3 e enp0s8), e lanciare la cattura sull'interfaccia sbagliata avrebbe prodotto un file .pcap vuoto o privo del traffico generato da Firefox. Abbiamo risolto controllando con attenzione l'output di `ip address` e scegliendo l'interfaccia con l'IP nella rete NAT (10.0.2.15), effettivamente collegata a Internet.

Un'altra piccola insidia e stata isolare il pacchetto corretto tra le centinaia catturati: senza i filtri `http` e `tcp.port==443`, la lista di Wireshark sarebbe stata illeggibile per la quantita di traffico di background (DNS, telemetria del browser, aggiornamenti). L'uso dei filtri di visualizzazione si e rivelato essenziale per arrivare rapidamente ai pacchetti di interesse.

09

Conclusioni

Questo esercizio ci ha permesso di vedere con i nostri occhi, byte per byte, la differenza tra un protocollo che fida ciecamente nella rete e uno che non fida di nessuno. Aver letto una password in chiaro dentro Wireshark con due click ci ha ricordato meglio di qualsiasi slide perché l'uso di HTTP per dati sensibili sia, di fatto, un invito a chiunque stia ascoltando sulla rete.

Allo stesso tempo, l'analisi della cattura HTTPS ci ha insegnato un limite importante: la cifratura protegge il canale, non il contenuto morale di chi sta dall'altra parte. Un sito malevolo con HTTPS resta malevolo, solo più difficile da intercettare per chi vuole rubare le credenziali via sniffing di rete. Per noi di NetRebels, l'insegnamento resta lo stesso di sempre: capire come funziona la sicurezza dal basso, pacchetto per pacchetto, e l'unico modo per non darla per scontata.

Documento redatto a scopo didattico — Cybersecurity & Ethical Hacking, EPICODE. Attività su VM Ubuntu in ambiente virtualizzato e controllato.